

Capstone Project - Progress Report - 1

Author: Sombaran Gupta **Email:** sombaran_pa2507mte92@iitp.ac.in **Date:** November 1, 2025 **Location:** Hyderabad, Telangana, India **Course:** EMTech - Cloud Computing **Subject Code:** EHS 5104 **Project Type:** Capstone project with CI/CD with Github Actions

Overview

ct-recon is a C++ project designed for modular, containerized development and deployment. It integrates modern C++ practices with a streamlined CI/CD pipeline using GitHub Actions and Docker. GitHub Actions is often chosen over Jenkins for modern CI/CD pipelines because it offers native GitHub integration, easier setup, and lower maintenance overhead—especially for cloud-native and open-source projects.

- The project is hosted on GitHub, so Actions integrates seamlessly.
- No need to manage Jenkins infrastructure.
- YAML workflows are version-controlled alongside the code.
- Docker and Conan steps are easily automated with reusable actions.
-

Ideal for a capstone project with limited deployment complexity.

Github Repository

Click here for view github repository

Key Technologies

Category	Tools/Technologies
Language	C++ (C++11 and beyond)
Build System	CMake (cmake version 3.22.1)
Package Manager	Conan (Conan version 2.22.1)
CI/CD	GitHub Actions
Containerization	Docker
Scripting	Shell, Python

Project Structure

- source/, include/, test/ – Core source and test files
- .github/workflows/ – GitHub Actions CI/CD definitions
- conanfile.py – Conan dependency management
- Dockerfile – Docker image definition
- run.sh – Local test script

```
2025-11-01 21:21:32 usomgupta@usomgupta ~/gitHub/ct-reconmain $ tree -L 3
.
├── CMakeLists.txt
├── CMakeUserPresets.json
├── Dockerfile
├── MODULE.bazel
├── README.md
├── build
│   ├── Release
│   │   ├── CMakeCache.txt
│   │   ├── CMakeFiles
│   │   ├── Makefile
│   │   ├── cmake_install.cmake
│   │   ├── ctReconServiceToolBin
│   │   ├── generators
│   │   ├── libctReconServiceTool.so
│   │   └── metadata
│   └── build
│       └── Release
├── conanfile.py
├── include
│   ├── BUILD
│   └── testInc.h
├── input.txt
├── mtechDocumentsSlides
├── run.sh
├── source
│   ├── BUILD
│   ├── CMakeLists.txt
│   ├── MODULE.bazel
│   ├── main.cpp
│   └── testSrc.cpp
├── test
│   ├── CMakeLists.txt
│   └── main.cpp
└── 11 directories, 22 files
```

CI/CD Workflow Diagram (PlantUML)

```
@startuml
title CI/CD Workflow for ct-recon

actor Developer

box "Build Preparation" #DDDDDD
    participant "Conan"
    participant "CMake"
end box
```

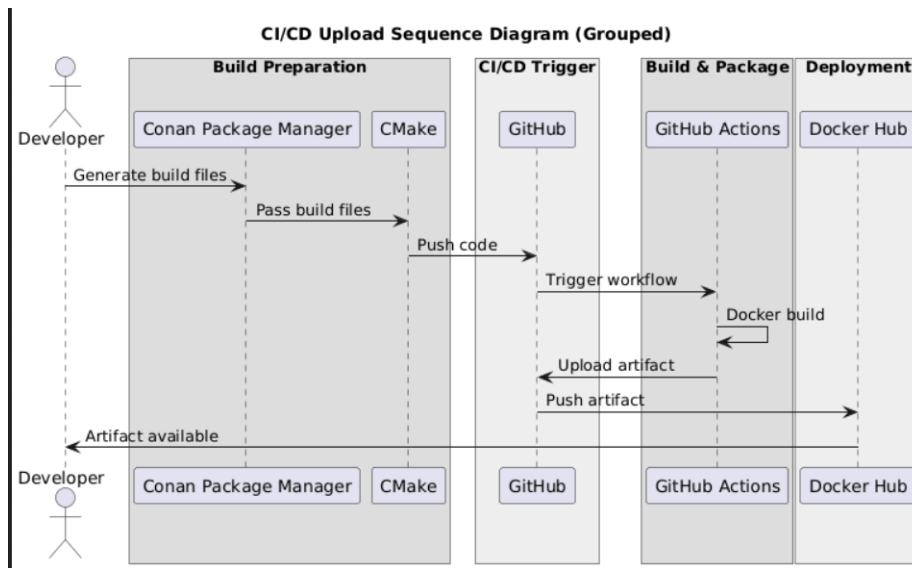


Figure 1: [Click here for sequence diagram](#)

```

box "Source Control" #EEEEEE
participant "GitHub"
end box

```

```

box "Automation" #DDDDDD
participant "GitHub Actions"
end box

```

```

box "Deployment" #EEEEEE
participant "Docker Hub"
end box

```

```

Developer -> Conan : Install dependencies
Conan -> CMake : Generate build files
CMake -> GitHub : Push source code
GitHub -> GitHub Actions : Trigger CI workflow
GitHub Actions -> GitHub Actions : Build Docker image
GitHub Actions -> Docker Hub : Push image
Docker Hub -> Developer : Image available for use

```

```

@enduml

```

Docker Usage

Build Docker image

```
docker build . -t ct-recon:<commit-hash-or-version>
```

Login to Docker Hub

```
docker login -u usomgupta
```

Run the container

```
docker run ct-recon:v1 ./ct-recon --input data.txt --verbose
```

Shell into the container

```
docker run -it [image ID] bash
```

Link to Docker Hub:

```
[Docker Hub: usomgupta/ct-recon] (https://hub.docker.com/r/usomgupta/ct-recon)
```

CI/CD Workflow

The CI/CD pipeline is triggered on code push and automates the build and deployment process using GitHub Actions and Docker Hub.

- Uses: ubuntu-latest image
- Checkout from repository
- Login into Docker Hub
- Set up Docker Build
- Build and push Docker image
- Upload build artifacts

name: CI for docker/build-push-action

on:

```
  push:
    branches:
      - main
```

permissions:

```
  contents: write
  issues: write
  pull-requests: write
```

jobs:

```
  build-and-push:
    runs-on: ubuntu-22.04 # Pin to a stable runner version
    permissions:
      contents: write
```

```

pull-requests: read

steps:
- name: Checkout repository
  uses: actions/checkout@v4
  with:
    fetch-depth: 1 # Shallow clone for faster checkout

- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    registry: ${ env.REGISTRY }
    username: ${ secrets.DOCKER_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }

- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3
  with:
    driver: docker-container
    driver-opts: |
      image=moby/buildkit:latest

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    builder: ${ steps.buildx.outputs.name }
    platforms: linux/amd64 # Use single platform unless multi-arch is needed
    context: .
    push: true
    tags: ${ secrets.DOCKER_USERNAME }/repo:v1
    cache-from: type=gha
    cache-to: type=gha,mode=max

- name: Upload C++ build artifacts
  uses: actions/upload-artifact@v4
  with:
    name: cpp-build-artifacts
    path: |
      build/libmylib.so
      build/myprogram

```

Local Testing

To run the project locally:

sh run.sh

Testing the docker artifact

```
PS C:\Users\ritup> docker pull usongupta/repo:v1
PS C:\Users\ritup> docker pull usongupta/repo:v1
v1: Pulling from usongupta/repo
d5c79deda2be: Pull complete
aa46819fa765: Pull complete
f8623a8386e7: Pull complete
8a422c469e9b: Pull complete
7d1b2d71253c: Pull complete
4f4fb700ef54: Pull complete
8e2f2dc427c8: Pull complete
888a6fd2a6da: Pull complete
236f3b202ef5: Pull complete
Digest: sha256:fa3d686e4c1391ab6db754bbadb76d5823d690d004933d7223846496d94b0074
Status: Downloaded newer image for usongupta/repo:v1
docker.io/usongupta/repo:v1
PS C:\Users\ritup> docker run -it bash^C
PS C:\Users\ritup> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
usongupta/repo      v1                 fa3d686e4c13       55 years ago       423MB
PS C:\Users\ritup> docker run -it fa3d686e4c13 bash
root@d708f1523a67:/app# ls
CMakeLists.txt      Dockerfile          README.md           conanfile.py        input.txt            run.sh              test
CMakeUserPresets.json  MODULE.bazel       build              include             mtechDocumentsSlides  source
root@d708f1523a67:/app# cd build/
root@d708f1523a67:/app# cd build/Release/
root@d708f1523a67:/app/build/Release# ls
CMakeCache.txt  CMakeFiles  Makefile  cmake_install.cmake  ctReconServiceToolBin  generators  LibctReconServiceTool.so  metadata
root@d708f1523a67:/app/build/Release# ./ctReconServiceToolBin
foo is called from main
Execution begin for functionreverseNumber
The reversed negative number 321
Execution begin for functionreverseNumber
The reversed negative number -321
root@d708f1523a67:/app/build/Release# cd ../../
root@d708f1523a67:/app# ls
CMakeLists.txt      Dockerfile          README.md           conanfile.py        input.txt            run.sh              test
CMakeUserPresets.json  MODULE.bazel       build              include             mtechDocumentsSlides  source
```

Figure 2: Gemini logo

Summary

The CI/CD pipeline is triggered on every code push. It automates build, test, and deployment using GitHub Actions, and publishes Docker images to Docker Hub for seamless distribution. Future work: **I will try to deploy artifacts in jfrog**
